

C 기초와 컴파일 과정

hello.c 가 실행파일이 되기까지, 그리고 그 안에서 값이 메모리에 놓이는 방식



오늘의 8교시

2시간 단위 4블록 · 각 교시는 예제 → 함정 → 빈칸 채우기 순서로 진행

1

개발환경·첫 컴파일

hello.c, 종료 코드

2

빌드 과정 해부

전처리·컴파일·어셈블·링크

3

헤더/소스 분리·Makefile

선언 vs 정의, 헤더 가드

4

자료형과 메모리

sizeof, 엔디안, 2의 보수

5

형변환·오버플로

$-1 < 1_u$, 정수 나눗셈

6

연산자·제어문

$\neq$, $\&/\&\&$, fall-through

7

함수와 스코프

값 전달, 스택 프레임, static

8

종합 실습

숫자 야구 · 정리

오전: 빌드 도구 오후: 언어 자체 마지막: 손에 익히기

준비 — 오늘 하루 종일 쓰는 한 줄

gcc 와 make 가 도는지 먼저 확인하고, 이 명령을 손에 익힙니다

```
$ gcc --version      # gcc (Ubuntu 13.x)
$ make --version     # GNU Make 4.x

$ gcc -Wall -Wextra -o app app.c && ./app
```

규칙 하나: 경고가 나오면 읽는다. 오늘 제출 조건은 -Wall -Wextra 경고 0개.

어제까지 Python으로 다룬 센서 프레임 $T=25.3, H=60, D=120^*$ 을 오늘부터 C로 다룹니다.

-Wall -Wextra

흔한 실수를 경고로. 오늘 함정 대부분이 여기서 잡힘

-o 이름

출력 파일 이름 (없으면 a.out)

-E / -S / -c

전처리·어셈블리·오브젝트에서 멈춤 (2교시)

-O0 / -O2

최적화 끄 / 켜

-g

gdb 디버깅 정보 (Day 4)

-lm

math.h 함수 링크 (5교시)

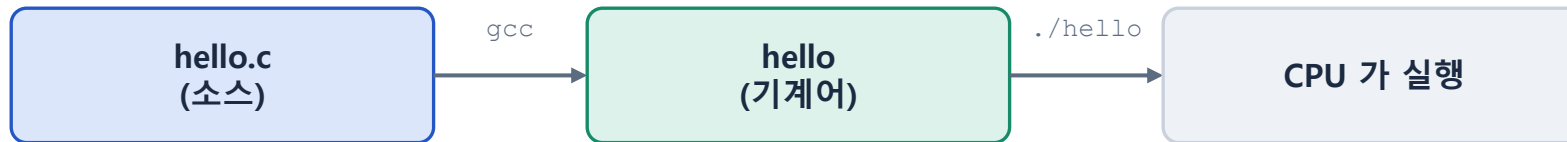
컴파일과 실행은 다른 일이다

1교시

```
#include <stdio.h>

int main(void) {
    printf("Hello, C!\n");
    return 0;
}
```

```
$ gcc hello.c
$ ls
a.out  hello.c
$ ./a.out
Hello, C!
$ gcc hello.c -o hello
$ ./hello
Hello, C!
```



Python과 다른 점: 소스를 고치면 다시 컴파일해야 결과가 바뀝니다. 문자열을 바꾸고 ./hello 만 다시 실행해 보세요 — 옛날 결과가 나옵니다.

main 의 반환값은 어디로 가는가 · 헤더를 빼면

1교시

종료 코드

```
int main(void) {  
    printf("센서 초기화 실패\n");  
    return 7;    // 0 이 아니면 비정상 종료  
}
```

```
$ ./exitcode; echo "종료 코드: $?"  
센서 초기화 실패  
종료 코드: 7
```

\$? 는 직전 프로그램의 종료 코드. 셸 스크립트·Makefile·systemd 가 이 값으로 성공/실패를 판단합니다. Jetson 서비스가 죽었는지 볼 때 다시 만납니다.

⚠ 함정 — #include 누락

```
// stdio.h 없음  
int main(void) {  
    printf("Hello\n");  
    return 0;  
}
```

```
$ gcc -Wall -Wextra nohdr.c  
error: implicit declaration of  
function 'printf'
```

헤더는 "이런 함수가 있다"는 선언을 담습니다. 선언이 없으면 컴파일러는 인자·반환 타입을 검사할 수 없습니다. 3교시에서 선언과 정의를 정확히 나눕니다.

gcc 한 줄 뒤의 네 단계

2교시

각 단계를 옵션으로 멈춰 세워 산출물을 직접 봅니다



`libc.a`
`printf, puts ...`

```
$ gcc -E hello.c -o hello.i      # ① 전처리
$ gcc -S hello.i -o hello.s      # ② 컴파일
$ gcc -c hello.s -o hello.o      # ③ 어셈블
$ gcc hello.o -o hello           # ④ 링크
$ wc -l hello.c hello.i         # 6 → 815
```

6줄이 815줄: `#include` 는 그 파일을 그
자리에 통째로 붙여넣기

🎬 D1-1 빌드 파이프라인 — 상자를 눌러 산출물 확인

산출물 읽기 — printf 는 어디로, puts 는 누가

2교시

hello.s (어셈블리)

```
.LC0:
    .string "Hello, C!"
main:
    pushq    %rbp
    movq     %rsp, %rbp
    leaq     .LC0(%rip), %rdi
    call     puts@PLT      # printf 가 아니라 puts
    movl     $0, %eax
    popq     %rbp
    ret
```

서식(%d 등)이 없고 \n 으로 끝나는 printf 는 컴파일러가 더 단순한 puts 로 바꿉니다. 코드를 "그대로" 번역하지 않는다는 첫 증거.

nm (심볼 표)

```
$ nm hello.o
                 U puts
0000000000000000 T main

$ nm hello | grep -E "main|puts"
0000000000001139 T main
                 U puts@GLIBC_2.2.5
```

- T** text — 이 파일에서 정의됨
- U** undefined — 누군가 채워야 함 (링커의 일)
- @GLIBC** 링크 완료 — 실행 시 libc.so 에서 찾을

⚠ 함정 — "컴파일은 됐는데 undefined reference"

2교시

```
#include <stdio.h>
int read_sensor(void); // 선언만, 본문 없음

int main(void) {
    printf("%d\n", read_sensor());
    return 0;
}
```

```
$ gcc -c link_err.c           # ③ 까지 성공
$ gcc link_err.o -o link_err # ④ 에서 실패
undefined reference to `read_sensor'
```

선언이 있으니 컴파일러는 만족. 정의를 어느 .o 에서도 못 찾았다는 링커의 말. 단골 원인: 함수 이름 오타 · .c 파일 빠뜨림 · -lm 누락

최적화 수준 비교

```
$ gcc -S -O0 hello.c -o o0.s
$ gcc -S -O2 hello.c -o o2.s
$ wc -l o0.s o2.s
  45 o0.s
  42 o2.s
$ diff o0.s o2.s # pushq %rbp 가 사라짐
```

- O0** 디버깅용. 코드를 그대로
- O2** 릴리스. 프레임 생략, tail call
- Os** 크기 우선 — MCU 플래시가 작을 때

선언은 .h, 정의는 .c

3교시

"무엇이 있는지" 와 "실제 본문" 을 나눈다

sensor.h

```
#ifndef SENSOR_H
#define SENSOR_H
typedef struct {
    double t; int h; int d;
} Reading;

int is_alert(Reading r);
void print_reading(Reading r);
#endif
```

sensor.c

```
#include <stdio.h>
#include "sensor.h"

int is_alert(Reading r) {
    return r.t > 35.0;
}

void print_reading(Reading r) {
    printf("T=%.1f %s\n", r.t,
        is_alert(r) ? "ALERT" : "OK");
}
```

main.c

```
#include "sensor.h"

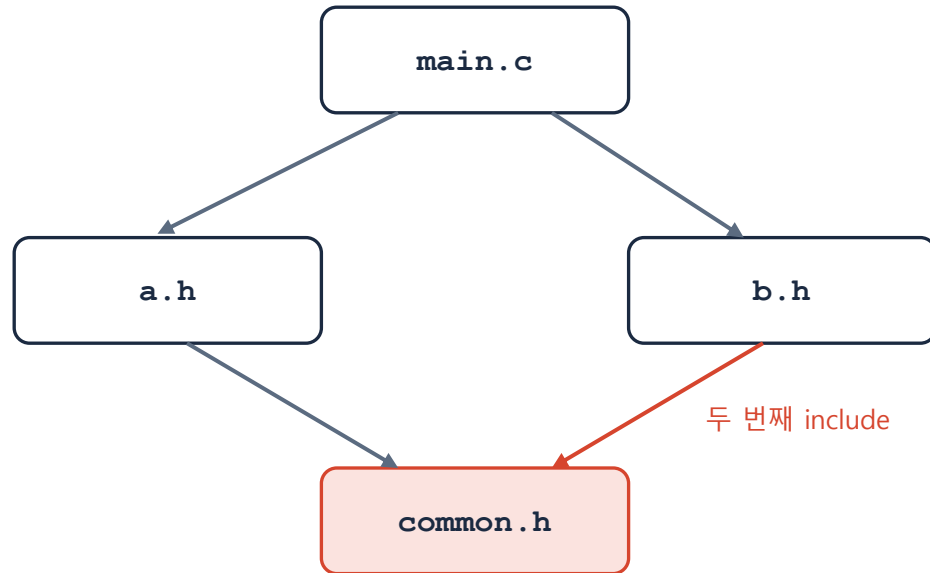
int main(void) {
    Reading r1 = {25.3, 60, 120};
    Reading r2 = {36.9, 70, 150};
    print_reading(r1);
    print_reading(r2);
    return 0;
}
```

```
$ gcc -Wall -Wextra main.c sensor.c -o app && ./app
T=25.3 OK
T=36.9 ALERT
```

"sensor.h" 따옴표: 현재 폴더 먼저 · <stdio.h> 꺾쇠: 시스템 경로만
구조체 문법은 Day 3 — 오늘은 "여러 값을 한 묶음으로" 로만 읽기

⚠ 함정 — 같은 헤더가 두 번 들어오면

3교시



해법: 헤더 가드

```
#ifndef COMMON_H    // 아직 정의 안 됐으면
#define COMMON_H    // 정의하고

typedef struct { int x, y; } Point;
int add(int a, int b);

#endif              // 이미 있으면 통째로 건너뛴
```

- 함수 선언 몇 번 나와도 OK
- 구조체·typedef 정의 한 번만. 두 번이면 redefinition 에러
- 가드 이름 파일명_H 대문자가 관례

```
// 전처리 결과 (가드 없음)
typedef struct {...} Point;    // a.h 경우
int add(int, int);
typedef struct {...} Point;    // b.h 경우 ← 재정의!
int add(int, int);            // 선언 중복은 OK
```

📺 D1-2 헤더 가드 — 토글을 끄고 에러부터 본다

Makefile — 바뀐 것만 다시 만든다

3교시

```
CC      = gcc
CFLAGS  = -Wall -Wextra -O2

app: main.o sensor.o
    $(CC) $(CFLAGS) -o app main.o sensor.o

main.o: main.c sensor.h
    $(CC) $(CFLAGS) -c main.c

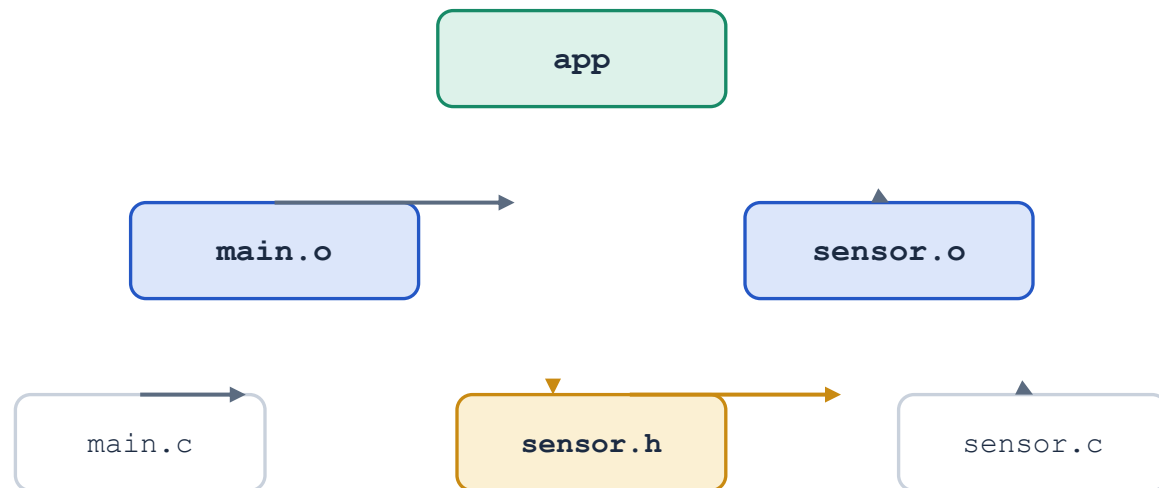
sensor.o: sensor.c sensor.h
    $(CC) $(CFLAGS) -c sensor.c

clean:
    rm -f *.o app
```

타깃: 의존파일 — 의존 파일이 타깃보다 새로우면 아래 명령 실행
명령 줄 앞은 반드시 탭 (공백이면 missing separator)

헤더를 의존성에 안 적으면: 헤더를 고쳐도 오래된 .o 가 링크되어 "
분명 고쳤는데 안 바뀌는" 버그

```
$ make          # gcc 세 줄 실행
$ make
make: 'app' is up to date.
$ touch sensor.c && make
# sensor.o, app 만 다시
$ touch sensor.h && make
# main.o, sensor.o, app 전부
```



자료형 = 바이트 수 + 해석 규칙

4교시

규칙: 예측 먼저 쓰고, 그 다음 실행

sizeof(...)	x86-64 Linux	ESP32 (32bit)
char	1	1
short	2	2
int	4	4
long	8	4 !
float	4	4
double	8	8
uint8_t	1	1
int32_t	4	4
int *	8	4 !

왜 고정폭 타입인가

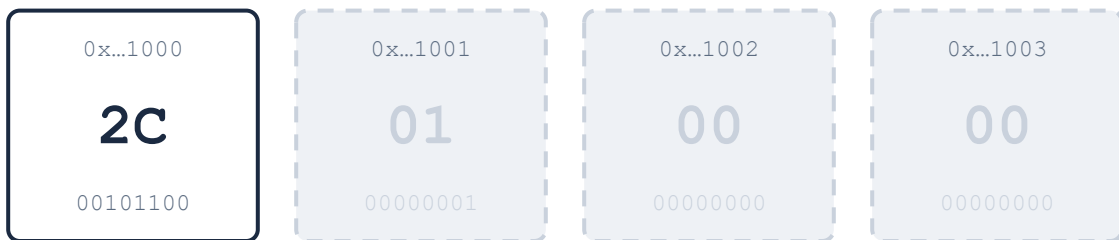
- long 은 Windows 에서 4, 포인터는 32비트 MCU 에서 4 — 플랫폼마다 다르다
- 레지스터·통신 프로토콜은 비트 폭이 정해져 있다
- 그래서 임베디드는 stdint.h 의 uint8_t · int16_t · uint32_t 를 쓴다
- %zu 는 size_t (sizeof 의 결과 타입) 출력용 서식

```
#include <stdint.h>
typedef struct {
    uint16_t distance_mm; // 0~4000
    int8_t    temp_c;      // -40~125
    uint8_t   humidity;    // 0~100
} Frame;                  // sizeof = 4
```

넘치면 잘린다 — 300 을 uint8_t 에 넣으면

4교시

```
int x = 300; // 0x0000012C
```



엔디안 — 바이트로 직접 보기

```
int v = 0x11223344;  
unsigned char *p = (unsigned char *)&v;  
printf("%02X %02X %02X %02X\n",  
        p[0], p[1], p[2], p[3]);  
// → 44 33 22 11
```

```
uint8_t a = 250; a += 10; // 260 → 4 (260-256)  
int8_t b = 127; b++; // → -128 (부호 비트로 넘어감)  
int x = 300; unsigned char c = x; // → 44 (0x2C)
```

```
printf("%d %d %d\n", a, b, c); // 4 -128 44
```

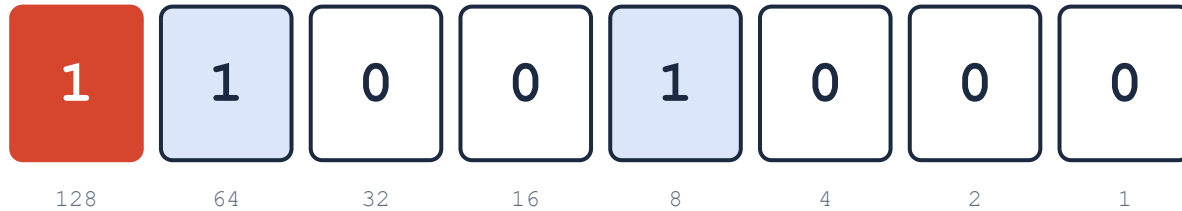
메모리 덤프나 시리얼로 받은 바이트가 "거꾸로" 보이는 이유.
네트워크 바이트 순서(빅 엔디안)와 반대라 프로토콜에서 항상 확인.

signed 오버플로는 C 표준상 정의되지 않은 동작 — 컴파일러가 뭘 해도 됨. 실제 코드에서 절대 의존 금지.

같은 비트, 다른 해석 — 2의 보수

4교시

부호 비트



unsigned char

200

signed char (2의 보수)

-56

$$128 + 64 + 8 = 200$$

$$200 - 256 = -56$$

```
unsigned char u = 200;
signed char   s = u;      // 같은 비트
printf("%u %d\n", u, s); // 200 -56
```

범위와 경계

`uint8_t` 0 ~ 255 $255 + 1 \rightarrow 0$

`int8_t` -128 ~ 127 $127 + 1 \rightarrow -128$

`uint16_t` 0 ~ 65535

`int32_t` $-2^{31} \sim 2^{31}-1$

127 = 01111111. 여기서 1을 더하는 순간 부호 비트로 넘어간다. 다이얼에서 바늘이 빨개지는 그 지점.

음수 128개 + 0 + 양수 127개 = 256. 그래서 -128은 있는데 +128은 없다.

📺 D1-4 2의 보수 다이얼 — 슬라이더 127 → 128

⚠ 함정 — -1 은 1 보다 작다. 정말?

5교시

-1 < 1u → false (0)

- 1 왼쪽은 int, 오른쪽은 unsigned int. 타입이 다르다
- 2 규칙: int 와 unsigned int 를 비교하면 int 쪽이 unsigned 로 변환된다
- 3 -1 의 비트 0xFFFFFFFF 를 unsigned 로 읽으면 4294967295
- 4 4294967295 < 1 은 거짓

```
printf("%d\n", -1 < 1);           // 1
printf("%d\n", -1 < 1u);          // 0
printf("%d\n", -1 < sizeof(int)); // 0
printf("%u\n", (unsigned)-1);     //
```

4294967295

```
$ gcc -Wall -Wextra promo.c
warning: comparison of integer
expressions of different
signedness [-Wsign-compare]
```

sizeof 의 결과가 size_t(unsigned) 라서, 배열 길이와 음수 인덱스를 비교하는 코드가 자주 빠지는 함정. -Wextra 가 잡아준다.

정수 나눗셈과 실수 비교

5교시

⚠ 정수 / 정수 = 정수

```
7 / 2      // 3      (버림)
7 / 2.0    // 3.5    (한쪽만 실수여도 실수 연산)
7 % 2      // 1      (나머지)

int c = 26;
double f1 = c * 9 / 5 + 32;    // 78.0   (234/5=46 잘림)
double f2 = c * 9.0 / 5 + 32;  // 78.8
```

c = 25 로 바꾸면 둘 다 77.0 으로 우연히 맞는다. 테스트 값 하나로 안심하면 안 되는 이유.

```
double avg = sum / 4;          // 정수면 잘림
double avg = (double)sum / 4;  // OK
double avg = (double)(sum / 4); // 틀림 - 잘린 뒤 변환
```

실수는 == 로 비교하지 않는다

```
#include <math.h>
double a = 0.1 + 0.2;
a == 0.3                                     // 0
printf("%.20f", a);                         // 0.300000000000000004441
fabs(a - 0.3) < 1e-9                       // 1   (오차 범위)
(int)3.99                                    // 3   (버림, 반올림 아님)
```

```
$ gcc -Wall -Wextra fp.c -o fp -lm
# -lm 을 빼면 → undefined reference to 'fabs'
#              = 2교시의 링크 에러
```

char 도 계산에 들어가면 int 로 승격. sizeof('A'+1) 은 4. uint8_t += 10 도 int 로 계산한 뒤 8비트로 잘라 넣는 것.

⚠ 함정 셋 — 전부 -Wall 이 경고한다

6교시

= vs ==

```
int x = 0;
if (x = 5) printf("A"); // A - 대입! 항상 참
if (x == 5) printf("B"); // B
```

습관: if (5 == x) 처럼 상수를 왼쪽에 — 실수하면 컴파일 에러

warning: suggest parentheses around assignment used as truth value

& vs &&

```
int a = 2, b = 4;
a & b // 0 비트 AND 010 & 100
a && b // 1 논리 AND 둘 다 0 아님
```

비트 연산은 Day 3 레지스터 제어의 핵심. 지금은 구분만.

시프트 우선순위

```
1 << 3 + 1 // 16 (+ 가 먼저: 1 << 4)
(1 << 3) + 1 // 9
```

시프트에는 항상 괄호. 레지스터 마스크 REG |= (1 << n) 습관.

warning: suggest parentheses around '+' inside '<<'

switch — 공은 벽(break)을 만날 때까지 굴러간다

6교시

```
int level = 2;
switch (level) {
    case 1: printf("FREEZE ");
    case 2: printf("COLD ");           // break 없음
    case 3: printf("NORMAL "); break;
    case 4: printf("WARM ");
    default: printf("ALERT ");
}
// → COLD NORMAL
```



case 1

case 2

case 3

case 4

default

n=2 진입 → case 3 의 break 에서 정지

```
// 의도적 fall-through 는 주석으로 표시
case 'y': case 'Y':           // 빈 case 는 경고 없음
    ok = 1; break;
case 1:
    prepare();
    // fall through           ← gcc 가 인식, 경고 사라짐
case 2:
    run(); break;
```

case 는 블록이 아니라 점프 목적지. level 을 1, 4, 9 로 바꿔 예측 → 실행.

```
warning: this statement may fall through
[-Wimplicit-fallthrough=]
```

🎬 D1-6 switch fall-through — 벽을 눌러 break 넣기

인자는 값이 복사된다 (call by value)

7교시

```
void inc(int x) {  
    x++;                // 복사본 x  
    printf("안: %d\n", x); // 안: 6  
}  
  
int main(void) {  
    int a = 5;  
    inc(a);  
    printf("밖: %d\n", a); // 밖: 5  
}
```

main 의 프레임

a = 5

inc 의 프레임

x = 5 → 6

복사

inc 가 끝나면 이 프레임은 사라지고, a 는 5 그대로

원본을 바꾸려면 주소를 넘겨야 한다 — Day 2 포인터의 첫 번째 동기.
오늘 세 번 미뤄둔 &v, const char *, &s 가 전부 여기서 풀린다.

지역 변수 → 스택 프레임 안 · 함수 반환 시 소멸

fact(3) — 프레임이 쌓이고 사라지는 순서

7교시

```
static int calls = 0; // 데이터 영역
```

```
int fact(int n) {  
    calls++;  
    if (n <= 1) return 1;  
    return n * fact(n - 1);  
}
```

```
int main(void) {  
    int r = fact(3); // 먼저 변수에  
    printf("결과 %d, 호출 %d번\n", r, calls);  
} // 결과 6, 호출 3번
```

스택 (아래→위)

fact(1) n=1 → 1

fact(2) n=2 → 2 · 1 = 2

fact(3) n=3 → 3 · 2 = 6

push ↑

pop ↓

데이터 영역

calls = 3

함수가 끝나도 남는다

- 들어갈 때 3→2→1, 돌아올 때 1→2→3 (LIFO)
- 가장 나중에 쌓인 프레임이 먼저 사라진다
- static · 전역은 스택과 무관하게 계속 산다

함수 안 static: 첫 호출에 한 번만 초기화, 호출 사이에 값 유지 (next_id 카운터, 이동평균 버퍼). 파일 범위 static: "이 파일 밖에서 안 보임" — 같은 단어, 다른 뜻.

⚠ 함정 — 인자 평가 순서 · 재귀의 한계

7교시

```
int next_id(void) { static int id = 0; return ++id; }

printf("%d %d %d\n", next_id(), next_id(), next_id());
// → 3 2 1 (gcc: 오른쪽부터 평가)

printf("결과 %d, 호출 %d번\n", fact(3), calls);
// → 결과 6, 호출 0번 ← calls 가 먼저 읽힘
```

C 표준은 인자 평가 순서를 정하지 않는다. 부수효과가 있는 호출을 한 식에 여러 번 넣지 말 것. 순서가 중요하면 변수에 먼저 받는다.

```
int depth(int n) {
    return n == 0 ? 0 : 1 + depth(n - 1);
}

depth(10000); // OK
depth(10000000); // Segmentation fault
```

스택 크기

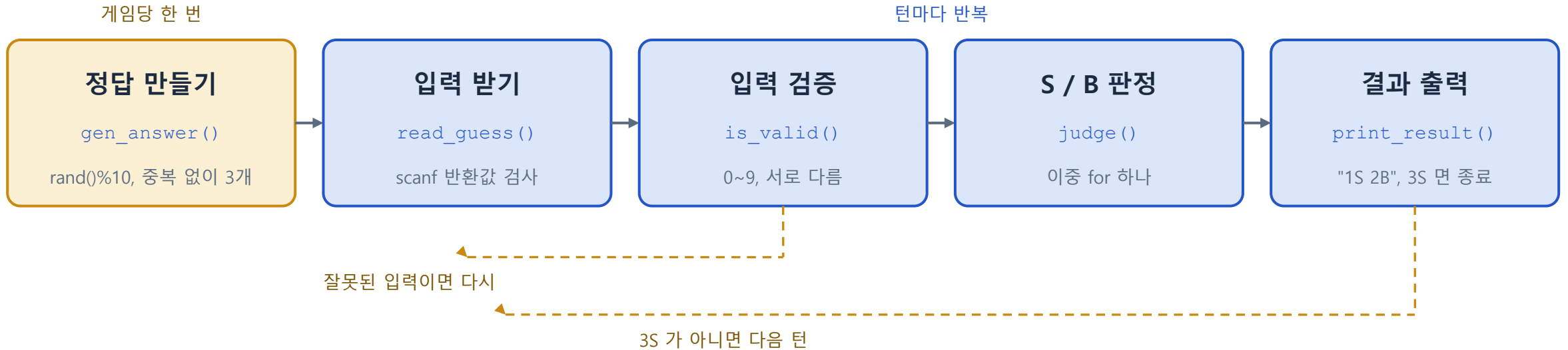
PC (Linux)	약 8 MB
ESP32 태스크	수 KB
Cortex-M0	1~2 KB

임베디드에서 재귀는 깊이를 확실히 알 때만.

종합 실습 — 숫자 야구, 함수로 쪼개기

8교시

파일 4개 (main.c · game.c · game.h · Makefile) · -Wall -Wextra 경고 0개



```
while (1) {
    if (!read_guess(g)) { printf("입력 오류\n"); continue; }
    if (!is_valid(g)) { printf("서로 다른 숫자 세 개\n"); continue; }
    tries++;
    int s = 0, b = 0;
    judge(ans, g, &s, &b); // &s - Day 2 에서 설명
    print_result(s, b, tries);
    if (s == 3) break;
}
```

뼈대(game.h · main.c · game.c 일부)는 실습 자료에 있습니다. is_valid, judge 의 ++, print_result 만 채우면 됩니다.
미완성이어도 함수 분리 구조가 잡혀 있으면 통과.

🎬 D1-8 숫자 야구 흐름 — 한 턴 재생

제출 체크리스트

8교시

☐ `make clean && make`

처음부터 빌드 성공

☐ `-Wall -Wextra`

경고 0개

☐ `"12" "abc" "1 1 2"`

잘못된 입력에서 죽지 않고 다시 묻는다

☐ `main.c game.c game.h Makefile`

파일 4개 — 선언은 .h 에, 정의는 .c 에

☐ `worksheet.html`

8교시까지 채점 → 결과 코드 · 오답 정리 캡처

```
$ ./baseball
세 자리 (예: 1 2 3) > 1 2 3
0S 1B
세 자리 (예: 1 2 3) > abc
입력 오류
세 자리 (예: 1 2 3) > 4 4 5
서로 다른 숫자 세 개
세 자리 (예: 1 2 3) > 7 3 9
3S 0B
정답! 2번 만에
```

제출 폴더: `c_day1/baseball/`

오늘의 여섯 문장

1

전처리 → 컴파일 → 어셈블 → 링크.
"undefined reference" 는 링크 에러다

 D1-1

2

선언은 .h, 정의는 .c.
헤더 가드로 재정의를 막고, Makefile 의존성에 헤더를 적는다

 D1-2

3

자료형 = 바이트 수 + 해석 규칙.
넘치면 잘리고, 같은 비트도 signed/unsigned 로 다르게 읽힌다

 D1-3 · D1-4

4

int 와 unsigned 를 섞으면 int 가 unsigned 로.
 $-1 < 1u$ 는 거짓

 D1-5

5

= 과 ==, & 와 &&, 시프트 괄호, switch 의 break.
네 함정은 전부 -Wall -Wextra 가 잡는다

 D1-6

6

인자는 값 복사, 지역 변수는 스택, static 은 데이터 영역.
재귀는 깊이를 알 때만

 D1-7

Day 2 예고

포인터

오늘 세 번 미뤄둔 것들이 전부 내일 첫 시간에 풀립니다

```
(unsigned char *)&v
```

4교시 · 바이트 단위로 보기

```
const char *level(double t)
```

6교시 · 문자열을 돌려주는 함수

```
judge(ans, g, &s, &b)
```

8교시 · 결과 두 개를 돌려받기

제출: c_day1/baseball/ (파일 4개) + worksheet.html 결과 캡처 · 내일 09:00